LeadTracker Pro — JavaScript Quiz

20 ахисан түвшний асуултын хариу ба дэлгэрэнгүй тайлбар

Өөрийн өмнөх хариу: 19 асуултаас 12 зөв, 1 асуулт хариултгүй үлдсэн. Доорх тайлбарууд зөв хариу болон яагаад гэдгийг алхамтайгаар тайлбарлана.

Хариуны түлхүүр

1-B 2-A 3-B 4-B 5-B 6-A 7-C 8-A 9-A 10-B
11-C 12-B 13-B 14-B 15-B 16-B 17-B 18-B 19-B 20-A

Тайлбар: “index” нь массив дахь байрлал, “ID” нь lead-ийг таних дугаар. Хайлт/filter хийсний дараа байрлал солигдож болох тул бодит төсөлд ID-аар олох нь найдвартай.

1. Search natijasi va index — B — Ali

filter() yangi, alohida array qaytaradi. Vali filtered array ichida index 0 bo‘lsa ham, asl savedLeads array’da Ali hanuz index 0 da. savedLeads[0] shu sabab Ali’ni beradi. Search natijasidagi index’ni asl array index’i deb ishlatish noto‘g‘ri leadni tanlashi mumkin.

```
filtered = [{ id: 20, name: "Vali" }] // filtered index 0
savedLeads[0] // Ali
```

2. Asl array’dagi indexni topish — A — findIndex()

findIndex() shartni qanoatlantirgan birinchi elementning asl array’dagi index’ini qaytaradi. ID 20 Vali obyektida bo‘lgani uchun natija 1. indexOf(20) mos emas, chunki array ichida 20 soni emas, obyektlar bor. length esa funksiya emas, sonli xususiyatdir.

```
const index = savedLeads.findIndex(lead => lead.id === 20);
// index === 1
```

3. findIndex topmasa — B — index === -1

findIndex() mos elementni topmasa -1 qaytaradi. Bu qiymatni tekshirish kerak, chunki -1 ni splice()ga berish array oxiridagi elementga ta’sir qilishi mumkin. null yoki undefined qaytmaydi.

```
if (index === -1) return;
savedLeads.splice(index, 1);
```

4. Nega if (!index) xavfli? — B — “Topilmadi” chiqadi

0 JavaScript’da falsy. Shuning uchun index 0 bo‘lsa, !index true bo‘lib qoladi. Ammo 0 — birinchi elementning haqiqiy index’i. Topilmadi holatini aniq -1 bilan solishtiring.

```
const index = 0;
!index // true
index === -1 // false
```

5. find() qaytargan object havolasi — B — Vali

find() topilgan objectning o‘ziga havola beradi, yangi nusxa yaratmaydi. selected.name o‘zgartirilganda leads[0] ko‘rsatgan o‘sha object ham o‘zgaradi. Shu sabab edit jarayonida topilgan leadni yangilash array ichidagi qiymatni yangilaydi.

```
const selected = leads.find(lead => lead.id === 1);
selected.name = "Vali";
leads[0].name // "Vali"
```

6. Object spread bilan nusxa — A — Ali

{ ...leads[0] } yangi object yaratadi. selected.name өөрчилсөндө leads[0] objectи unchanged qoladi. Bu shallow copy: ichkarida nested object bo‘lsa, uning havolasi alohida nusxalanmasligi mumkin.

```
const selected = { ...leads[0] };
selected.name = "Vali";
leads[0].name // "Ali"
```

7. filter() asl arrayni o‘zgartiradimi? — C — 2

filter() natijani yangi array qilib qaytaradi; asl arrayni o‘zgartirmaydi. Natija o‘zgaruvchiga berilmagani uchun u yo‘qoladi, savedLeads esa ikkita element bilan qoladi. O‘chirish uchun qaytgan arrayni qayta tayinlang yoki splice ishlating.

```
savedLeads.filter(lead => lead.id !== 1);
savedLeads.length // 2

savedLeads = savedLeads.filter(lead => lead.id !== 1);
savedLeads.length // 1
```

8. splice() natijasi — A — removed ["Vali"], qolganlar ["Ali", "Said"]

splice(1, 1) index 1 dan boshlab bitta elementni asl array’dan olib tashlaydi. Olib tashlangan element(lar)ni alohida array qilib qaytaradi. Demak removed = ["Vali"], savedLeads esa ["Ali", "Said"] болно.

```
const removed = savedLeads.splice(1, 1);
// removed: ["Vali"]
// savedLeads: ["Ali", "Said"]
```

9. Edit branch дахь return — A — шинэ lead давхар үүсч болно

Save handler editId байгаа үед одоо байгаа lead-ийг шинэчилнэ. return байхгүй бол гүйцэтгэл доош үргэлжилж, шинээр object үүсгэдэг хэсэгт хүрч болзошгүй. return нь edit амжилттай дууссаны дараа handler-ийг шууд зогсооно.

```
if (editId !== null) {
updateExistingLead();
return; // new-lead хэсэг рүү оруулахгүй
}
createNewLead();
```

10. localStorage.getItem ба төрөл — B — String

localStorage зөвхөн текстэн утга хадгална. JSON.stringify array-г JSON текст болгоно. getItem() энэ текстийг буцаадаг тул JSON.parse() хийж байж JavaScript array/object болгож ашиглана.

```
const data = localStorage.getItem("savedLeads");
typeof data // "string"
const leads = JSON.parse(data);
```

11. Refresh дараа өгөгдөл — C — Lead алга болно

savedLeads нь JavaScript ой санамж дахь массив. Страниц шинэчлэгдэхэд скрипт дахин эхэлж массив []-оос эхэлнэ. render() зөвхөн DOM-ийг зурна, хадгалалт хийхгүй. Удаан хадгалахын тулд өөрчлөлт бүрийн дараа localStorage.setItem хэрэгтэй.

```
savedLeads.push(lead);
render(savedLeads, saveLead);
// saveLeadsToStorage байхгүй => refresh-д array дахин хоосон
```

12. Favorite рүү шилжүүлэх дараалал — B — Эхлээд array-уудыг өөрчил, дараа нь хадгал

Storage-д хамгийн сүүлийн төлөв бичигдэх ёстой. Эхлээд lead-ийг favLeads-д нэмж, savedLeads-ээс хасна. Дараа нь хадгалж, хоёр хэсгийг render хийнэ. Хадгалалтыг өөрчлөлтөөс өмнө хийвэл хуучин төлөв үлдэнэ.

```
favLeads.push(lead);
savedLeads.splice(index, 1);
saveLeadsToStorage();
render(savedLeads, saveLead);
render(favLeads, favDbl);
```

13. || ба хоёр массив дахь хайлт — B — Эхний олдсон lead буцна

|| эхний truthy утгыг буцаадаг. savedLeads.find() object олбол тэр object truthy тул favLeads.find() ажиллах шаардлагагүй. Эхнийх undefined бол баруун талын хайлт хийгдэнэ.

```
const lead = savedLeads.find(match) || favLeads.find(match);
```

14. Шаардлагатай талбарууд — B — || ашиглах

Мессеж гаргах нөхцөл нь “талбаруудаас дор хаяж нэг нь хоосон”. Иймд || зөв: аль нэг хоосон бол нөхцөл true. && хэрэглэвэл зөвхөн бүгд зэрэг хоосон үед л true болно.

```
if (name === "" || email === "") {
alert("Fill in fields");
}
```

15. trim ба whitespace — B — “Ism kiriting” хэвлэгдэнэ

" " нь хоосон string биш, харин зай тэмдэгтүүдтэй string. trim() эхлэл ба төгсгөлийн хоосон зайг авч хаяж, "" болгоно. Ийм шалгалт зөвхөн space дарж form илгээхийг хориглоно.

```
const name = " ";
name.trim() === "" // true
```

16. Event delegation — B — Шинэ card-уудтай ажиллана

Handler байнгын parent container дээр байна. Card-ууд innerHTML-ээр устаж дахин үүссэн ч parent өөрөө хэвээр тул bubbling болсон event-ийг авна. closest() ашиглаж event аль card-аас ирснийг танина.

```
saveLead.addEventListener("dblclick", event => {
const card = event.target.closest(".lead-card");
});
```

17. innerHTML шинэчлэлт ба event handler — B — Шинэ card хуучин handler авахгүй

innerHTML өөрчлөхөд хуучин хүүхэд элементүүд DOM-оос хасагдаж, шинэ element-үүд бий болно. Хуучин card дээр тусад нь тавьсан handler шинэ card-д хуулбарлагдахгүй. Шинэ event listener холбох эсвэл event delegation хэрэгтэй.

```
card.addEventListener("click", handler);
container.innerHTML = "<div class=lead-card>New</div>";
// шинэ div-д хуучин handler алга
```

18. Том/жижиг үсгийн хайлт — B — хоёр талыг lower-case болгох

includes() үсгийн том-жижгийг ялгадаг. "Saidjon" дотор яг "said" гэсэн хэсэг байхгүй гэж үзнэ, учир нь эхний S том. Хоёр мөрийг toLowerCase() болговол хайлт case-insensitive болно.

```
name.toLowerCase().includes(search.toLowerCase()) // true
```

19. Dashboard нийт тоо — B — хоёр length-ийг нэмэх

Array-уудын өөрсдийг нь нэмэх нь тоо гаргахгүй. length нь массивын элементүүдийн тоо. Танай логикт lead saved эсвэл favorite массивын зөвхөн нэгд нь байдаг тул уртуудын нийлбэр нийт lead болно.

```
const total = savedLeads.length + favLeads.length;
```

20. Нэг ижил ID-г устгах — A — ижил ID-тай хоёр lead хоёулаа алга болно

filter() бүх элементийг шалгана. id нь leadId-тэй тэнцүү бол item.id !== leadId false болж, тэр элемент үр дүнд орохгүй. Ижил ID хоёр object дээр давтагдвал хоёулаа хасагдана; ID өвөрмөц байх ёстой.

```
savedLeads = savedLeads.filter(lead => lead.id !== leadId);
```

Гол ойлголтуудыг нэгтгэв

• find: тохирох элементийн утгыг буцаана; findIndex: тухайн элементийн байрлалыг буцаана; олоогүй бол -1.

• filter: шинэ массив үүсгэнэ; splice: одоогийн массивыг өөрчилнө.

• render: DOM-ийг шинэчилнэ; localStorage: refresh хооронд мэдээллийг хадгална.

• Search: filter хийсний дараах index-ийг эх массивын index гэж бүү үз; ID ашиглан lead-ийг ол.